A B S C HLUSSPROJEKT

# LaserFlow

## Use Cases, Wireframes und Technologie Stack

Von Sita Stojan, Marcus Gonzalez und Mario Lindner | 01.10.2026

# Inhaltsverzeichnis

**1 Use Cases und Wireframes ..... 3**

1.1 Übersicht der Akteure und Use Cases ..... 3

1.2 Besucher ..... 5

1.3 Kunde: Bild hochladen ..... 9

1.4 Kunde: Warenkorb und Bestellung ..... 11

1.5 Kunde: Bestellhistorie und persönliche Daten ..... 13

1.6 Mitarbeiter ..... 15

1.7 Admin ..... 17

**2 Technologie Stack ..... 18**

2.1 Anwendungsart ..... 18

2.2 Programmiersprachen und Frameworks ..... 18

2.3 Datenhaltung ..... 19

2.4 Architektur ..... 20

# 1 Use Cases und Wireframes

Die Use-Case-Diagramme zeigen, welche Akteure welche Funktionen von LaserFlow nutzen. Das erste Diagramm gibt den Überblick, danach folgt je ein Diagramm pro Akteur und Ablauf mit den enthaltenen Teilschritten (include) und optionalen Erweiterungen (extend).

Die angefügten Wireframes dienen zur Visualisierung der Funktionen des Use-Case-Diagrammes.

## 1.1 Übersicht der Akteure und Use Cases

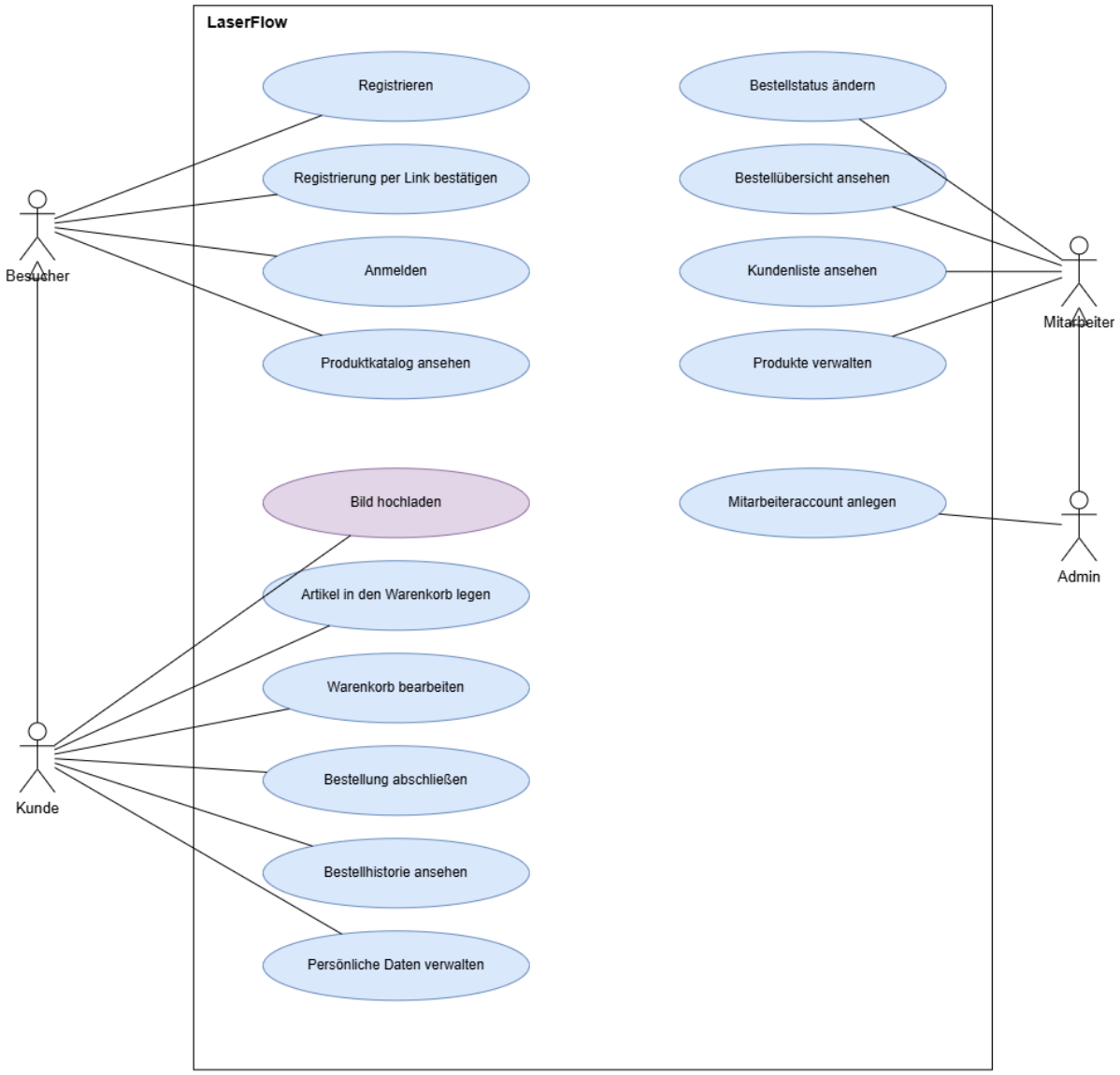



*Abbildung 1: Use-Case-Übersicht*

### Akteure

| Akteur | Beschreibung |
|---|---|
| **Besucher** | Nicht angemeldete Person. Sieht Katalog und Produktdetails, kann sich registrieren und anmelden. |

| | |
|---|---|
| **Kunde** | Angemeldeter Besucher (erbt alle Use Cases des Besuchers). Lädt Bilder hoch, bestellt und verwaltet sein Konto. |
| **Mitarbeiter** | Bearbeitet Bestellungen, verwaltet Produkte, sieht die Kundenliste. |
| **Admin** | Erbt alle Use Cases des Mitarbeiters und verwaltet zusätzlich Mitarbeiteraccounts und Rollen. |

### Use Cases im Überblick

- **Besucher:** Produktkatalog ansehen, Registrieren, Registrierung per Link bestätigen, Anmelden
- **Kunde:** Bild hochladen, Artikel in den Warenkorb legen, Warenkorb bearbeiten, Bestellung abschließen, Bestellhistorie ansehen, Persönliche Daten verwalten
- **Mitarbeiter:** Bestellübersicht ansehen, Bestellstatus ändern, Kundenliste ansehen, Produkte verwalten
- **Admin:** Mitarbeiteraccount anlegen

## 1.2 Besucher

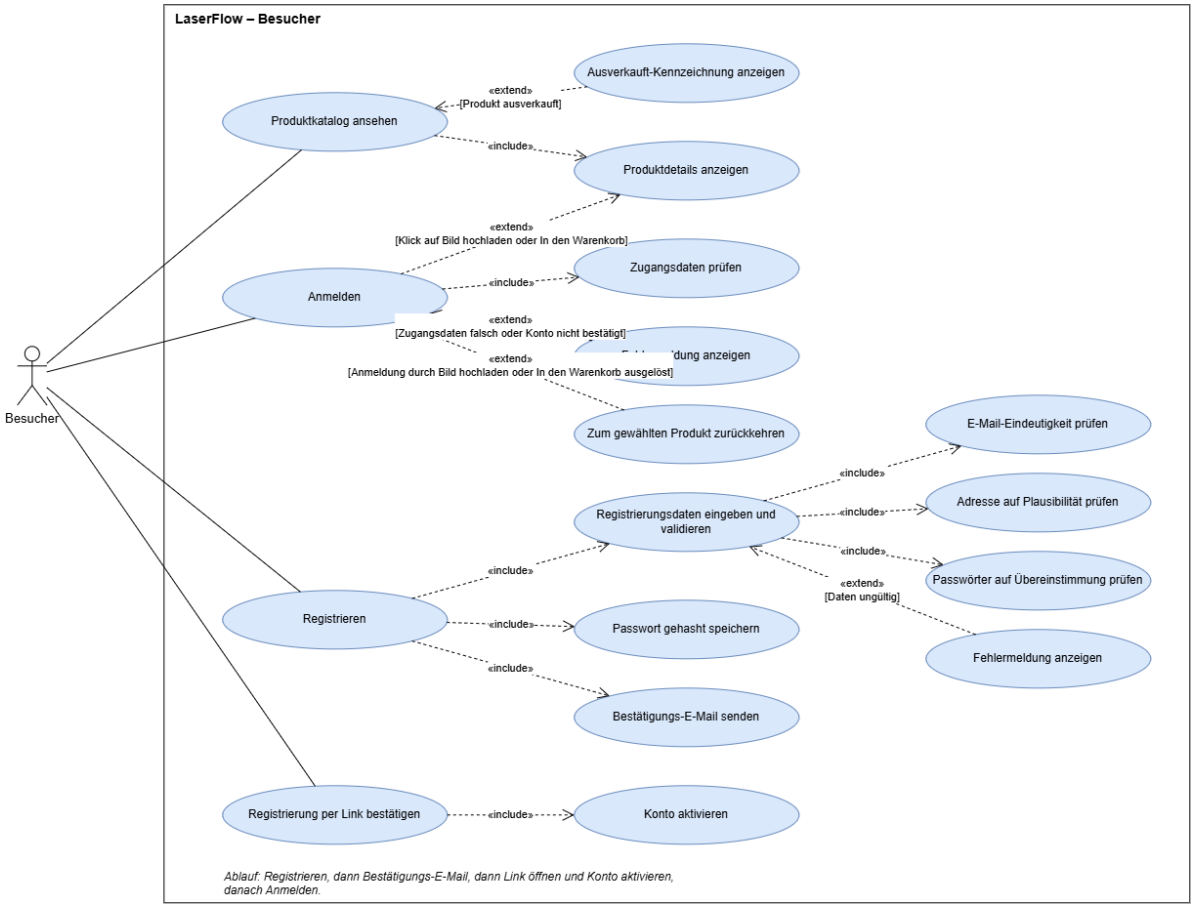
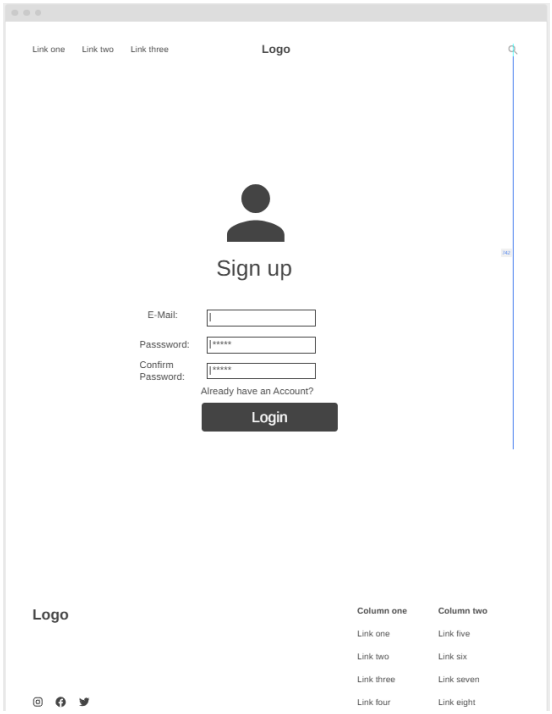


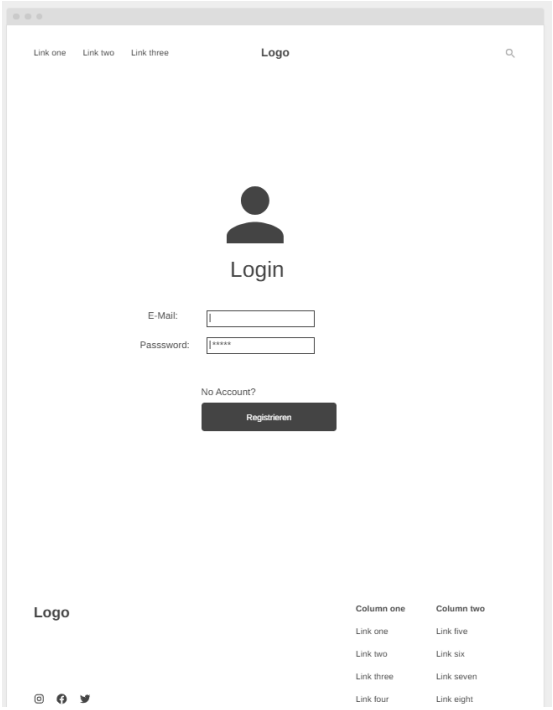

*Abbildung 2: Use-Case-Diagramm Besucher*

### Use Cases

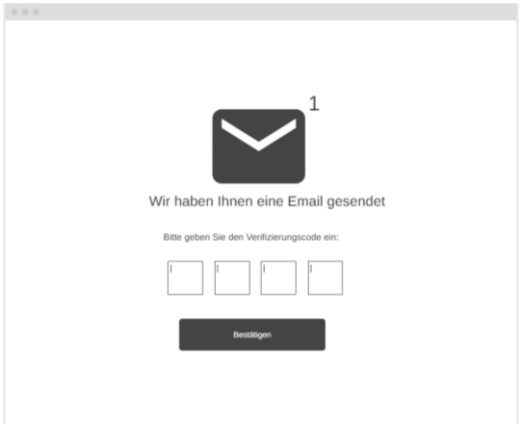
- Produktkatalog ansehen (Produktdetails anzeigen, Ausverkauft-Kennzeichnung anzeigen)
- Anmelden (Zugangsdaten prüfen, Fehlermeldung anzeigen, Zum gewählten Produkt zurückkehren)
- Registrieren (Registrierungsdaten eingeben und validieren, Passwort gehasht speichern, Bestätigungs-E-Mail senden)
- Registrierung per Link bestätigen (Konto aktivieren)



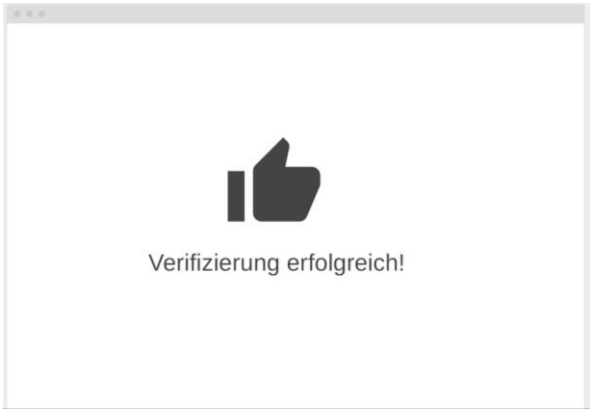

Registrierung



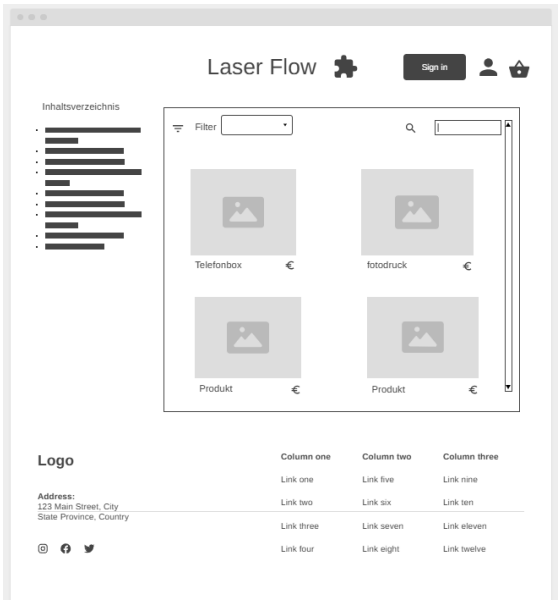

Login



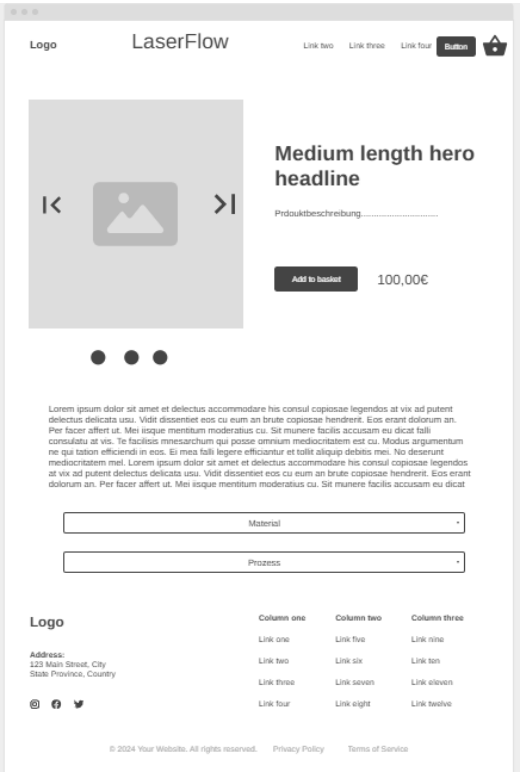

Verifizierungsvorgang




Bestätigung der Verifizierung




Produktkatalog anzeigen




Produktdetails anzeigen

## 1.3 Kunde: Bild hochladen

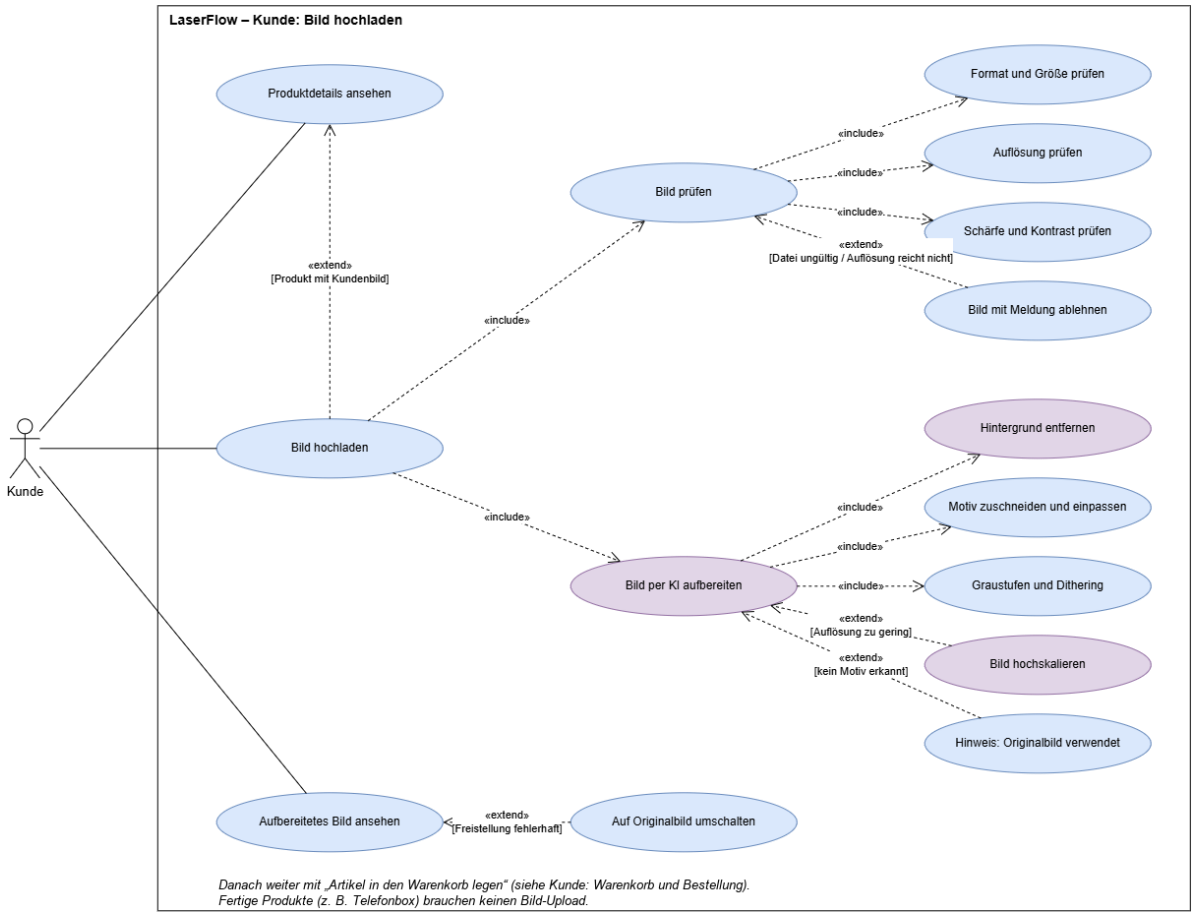
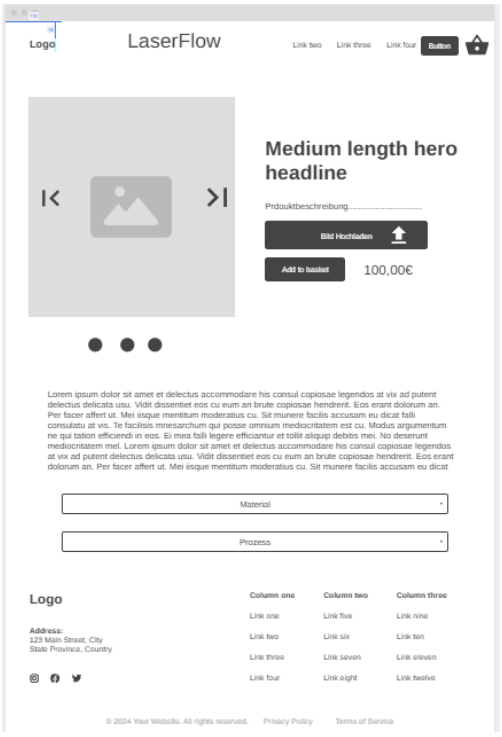


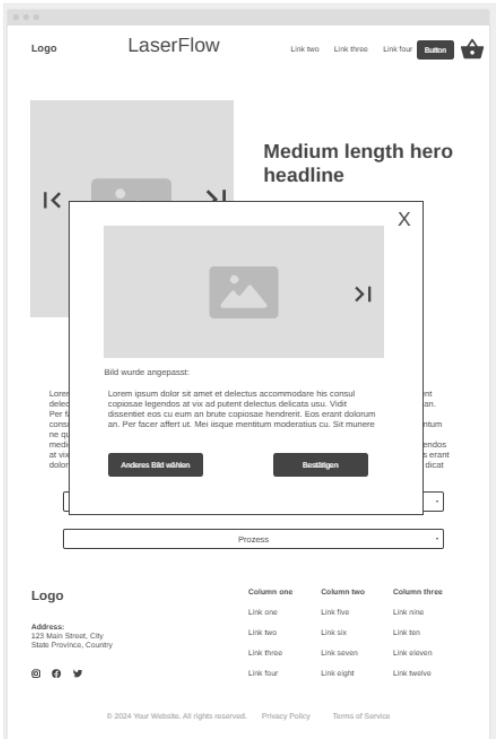

*Abbildung 3: Use-Case-Diagramm Kunde: Bild hochladen*

### Use Cases

- Produktdetails ansehen
- Bild hochladen (Bild prüfen, Bild per KI aufbereiten)
- Bild prüfen: Format und Größe, Auflösung, Schärfe und Kontrast prüfen; Bild mit Meldung ablehnen
- Bild per KI aufbereiten: Hintergrund entfernen, Motiv zuschneiden und einpassen, Graustufen und Dithering; Bild hochskalieren; Hinweis „Originalbild verwendet"
- Aufbereitetes Bild ansehen (Auf Originalbild umschalten)




Produkt-Bild hochladen




Aufbereitetes Bild ansehen

## 1.4 Kunde: Warenkorb und Bestellung

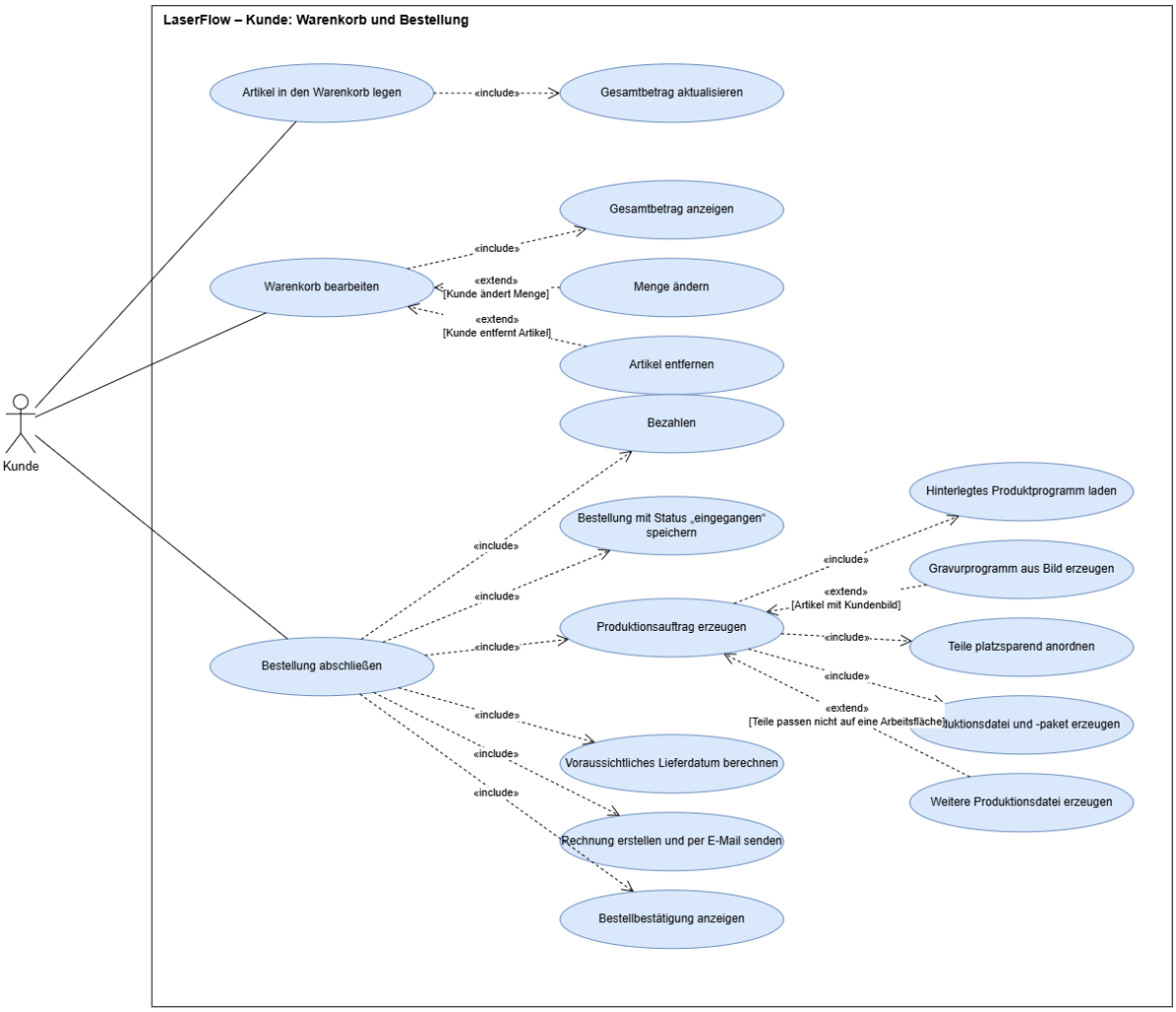
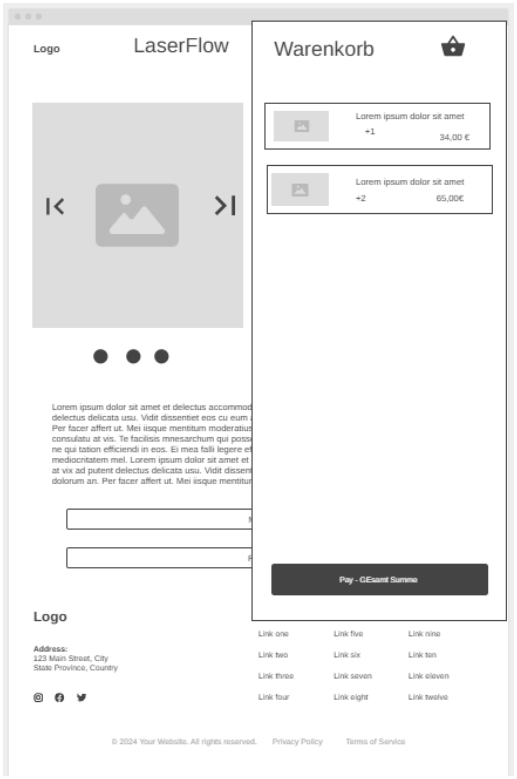


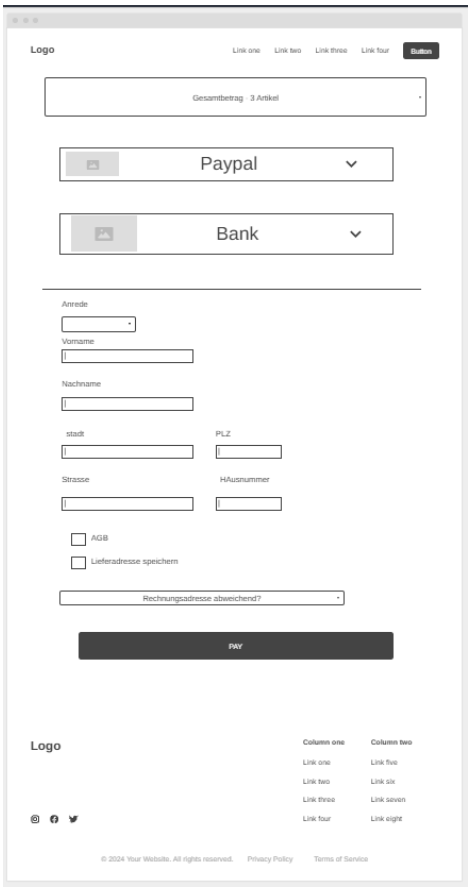

*Abbildung 4: Use-Case-Diagramm Kunde: Warenkorb und Bestellung*

### Use Cases

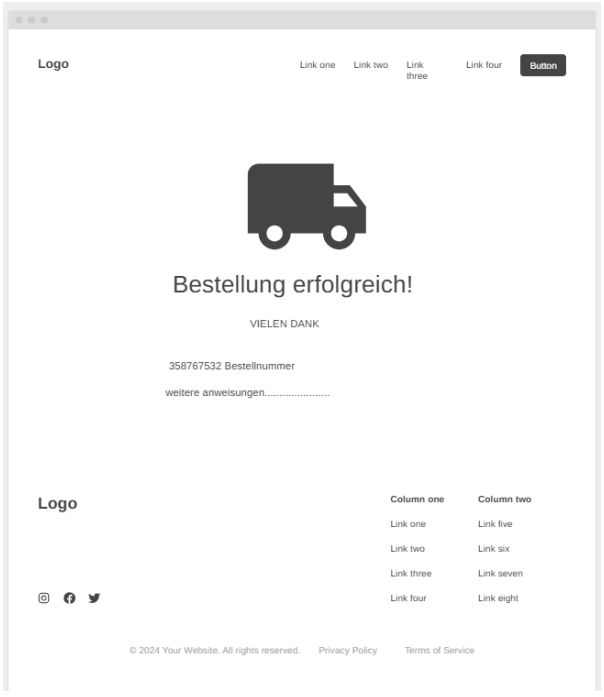
- Artikel in den Warenkorb legen (Gesamtbetrag aktualisieren)
- Warenkorb bearbeiten (Gesamtbetrag anzeigen, Menge ändern, Artikel entfernen)
- Bestellung abschließen: Bezahlen, Bestellung mit Status „eingegangen“ speichern, Produktionsauftrag erzeugen, Voraussichtliches Lieferdatum berechnen, Rechnung erstellen und per E-Mail senden, Bestellbestätigung anzeigen
- Produktionsauftrag erzeugen: Hinterlegtes Produktprogramm laden, Gravurprogramm aus Bild erzeugen, Teile platzsparend anordnen, Produktionsdatei und -paket erzeugen, Weitere Produktionsdatei erzeugen




Warenkorb ansehen und bearbeiten




Bestellung abschließen




Bestellbestätigung anzeigen

## 1.5 Kunde: Bestellhistorie und persönliche Daten

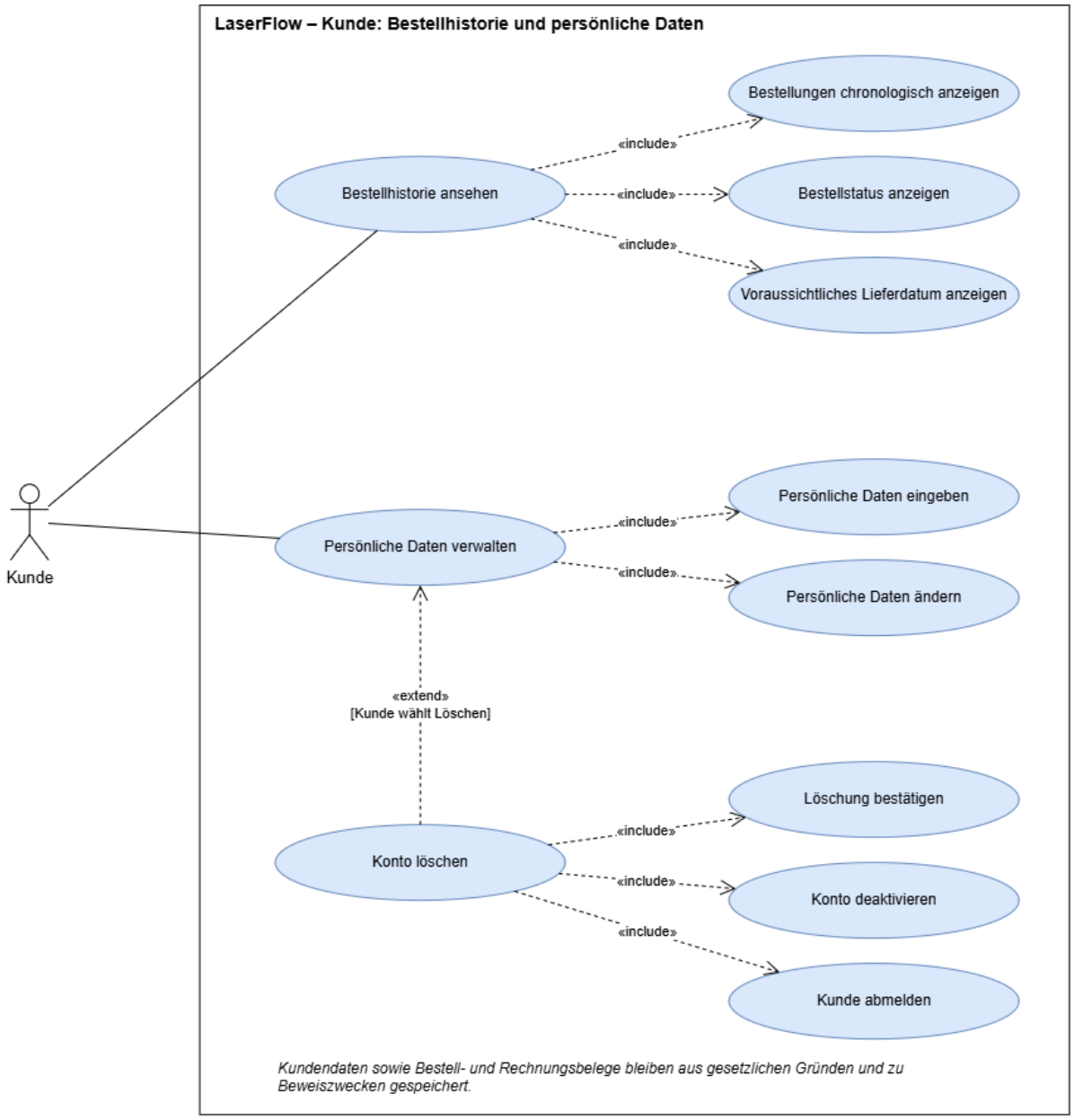
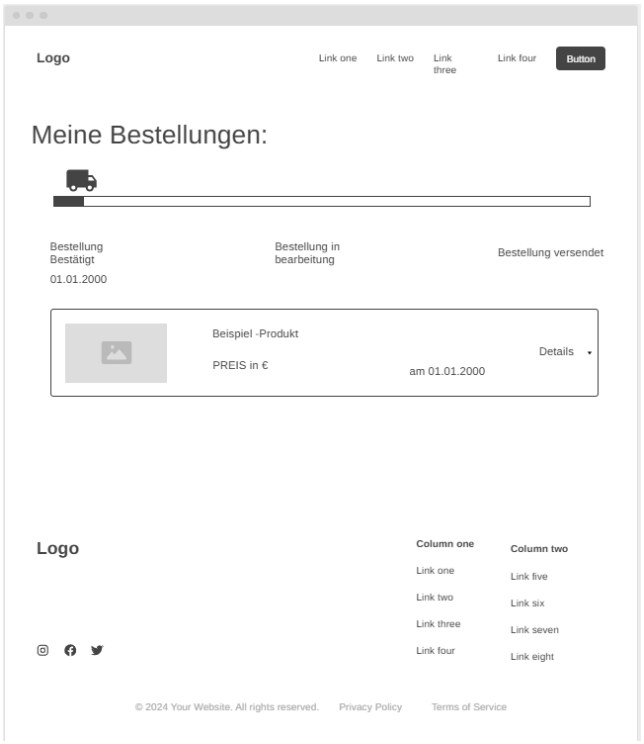



*Abbildung 5: Use-Case-Diagramm Kunde: Bestellhistorie und persönliche Daten*

### Use Cases

- Bestellhistorie ansehen (Bestellungen chronologisch anzeigen, Bestellstatus anzeigen, Voraussichtliches Lieferdatum anzeigen)
- Persönliche Daten verwalten (Daten eingeben, Daten ändern)
- Konto löschen (Löschung bestätigen, Konto deaktivieren, Kunde abmelden)




Bestellhistorie anzeigen und Bestellstatus anzeigen




Persönliche Daten verwalten

## 1.6 Mitarbeiter

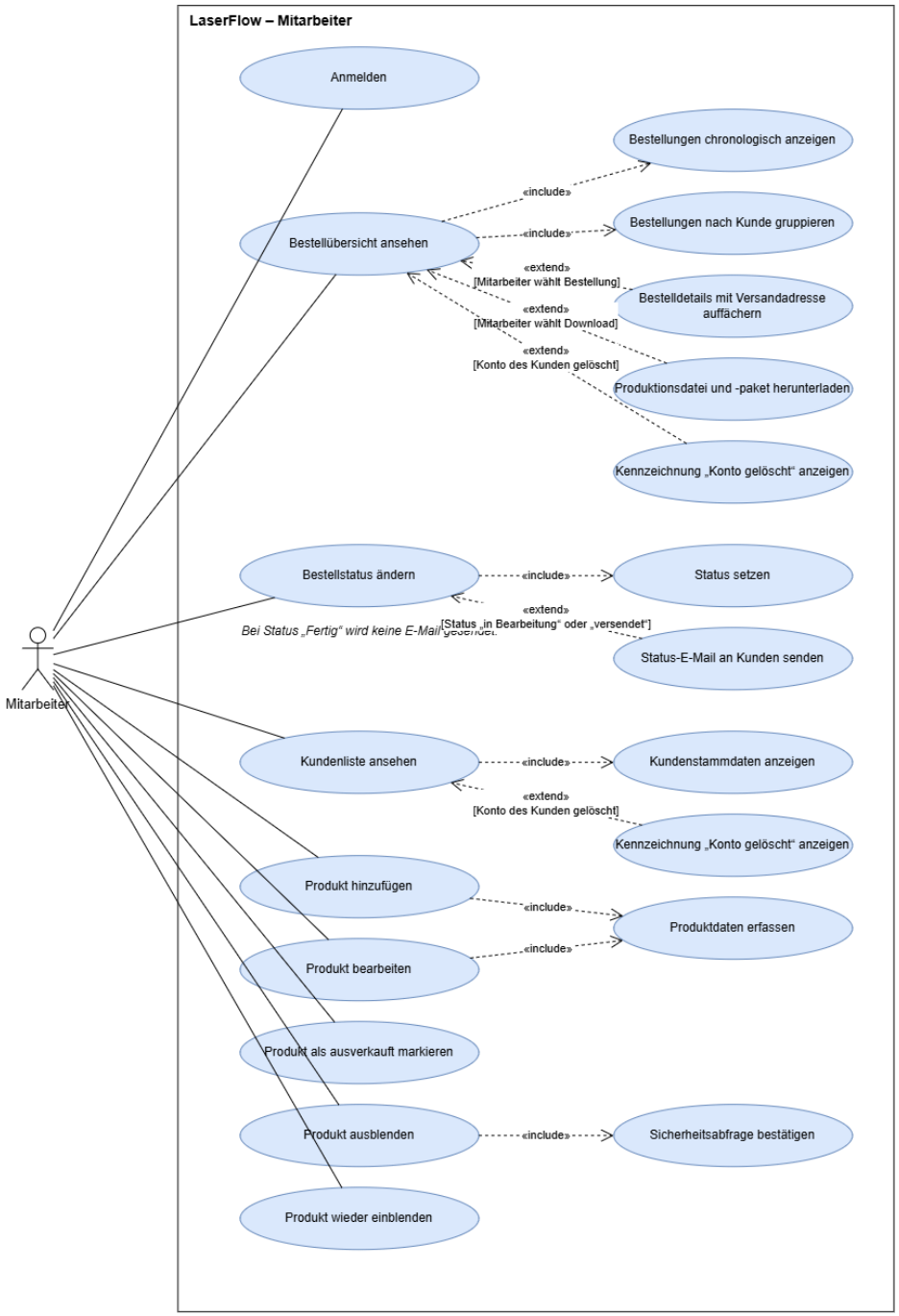
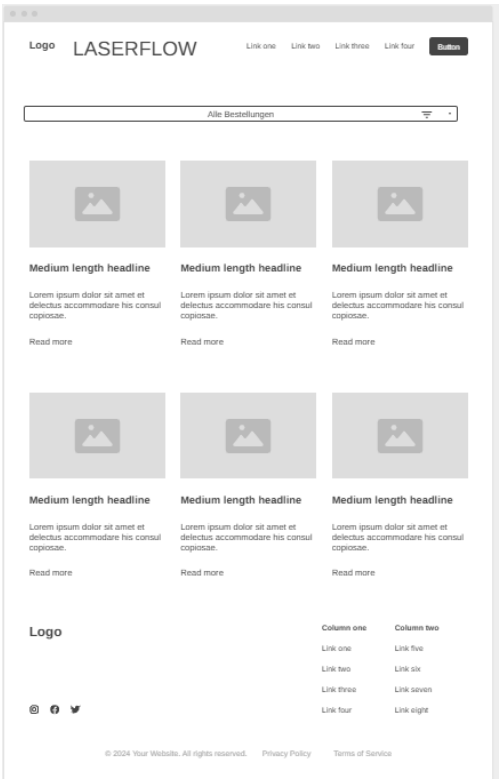


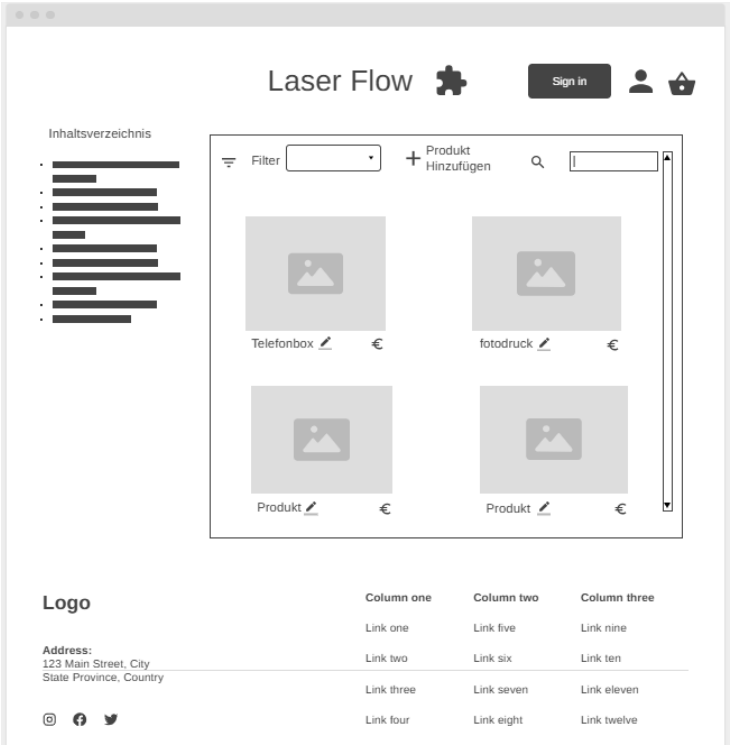

*Abbildung 6: Use-Case-Diagramm Mitarbeiter*

### Use Cases

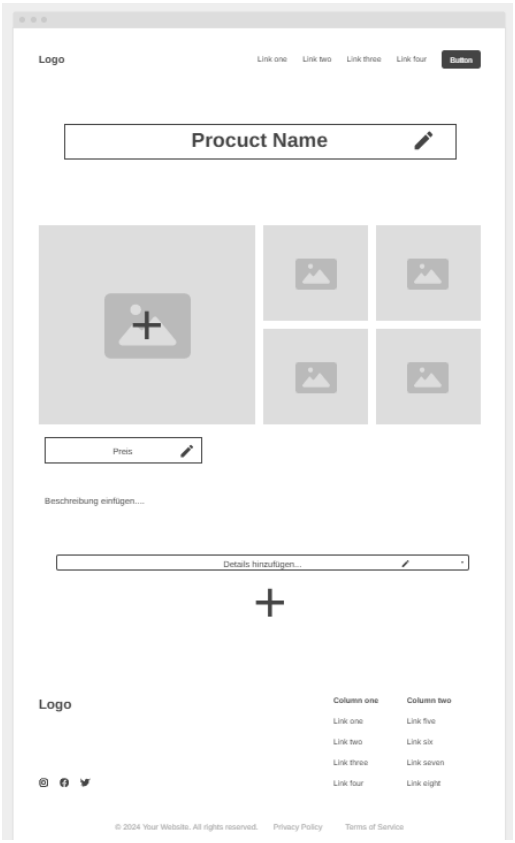
- Anmelden
- Bestellübersicht ansehen (Bestellungen chronologisch anzeigen, Bestellungen nach Kunde gruppieren, Bestelldetails mit Versandadresse auffächern, Produktionsdatei und -paket herunterladen, Kennzeichnung „Konto gelöscht“ anzeigen)
- Bestellstatus ändern (Status setzen, Status-E-Mail an Kunden senden)
- Kundenliste ansehen (Kundenstammdaten anzeigen, Kennzeichnung „Konto gelöscht“ anzeigen)
- Produkt hinzufügen, Produkt bearbeiten (Produktdaten erfassen), Produkt als ausverkauft markieren, Produkt ausblenden (Sicherheitsabfrage bestätigen), Produkt wieder einblenden




Bestellübersicht ansehen




Produkte bearbeiten




Produkt hinzufügen




Kundenliste ansehen

## 1.7 Admin

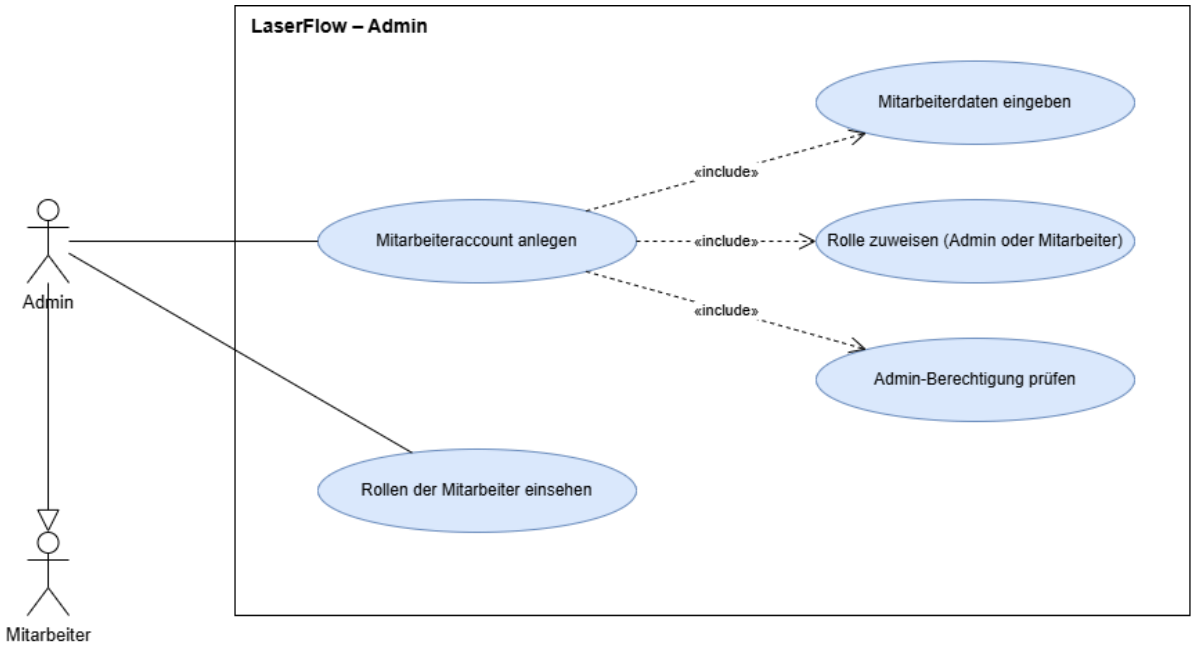



*Abbildung 7: Use-Case-Diagramm Admin*

### Use Cases

- Mitarbeiteraccount anlegen (Mitarbeiterdaten eingeben, Rolle zuweisen, Admin-Berechtigung prüfen)
- Rollen der Mitarbeiter einsehen
- Der Admin erbt alle Use Cases des Mitarbeiters.




Mitarbeiter anlegen und bearbeiten

# 2 Technologie Stack

## 2.1 Anwendungsart

| Option | Vorteile | Nachteile |
|---|---|---|
| **Statische Website** | Geringster Aufwand, schnelle Ladezeiten | Kein Login, kein Upload, keine Bestellung |
| **Web-App** | Ohne Installation, eine Codebasis für alle | Online-Betrieb erfordert Internetverbindung |
| **Mobile App (iOS, Android)** | Voller Zugriff auf Gerätefunktionen | Store-Gebühren, oft zwei Codebasen |
| **Desktop-Anwendung** | Offline nutzbar, Zugriff auf lokale Dateien | Installation beim Kunden, Updates je System |

> **Behauptung:** Die Web-App ist im Vergleich zu statischer Website, Mobile App und Desktop-Anwendung die beste Wahl, weil sie alle Muss-Anforderungen mit nur einer Entwicklung für alle gängigen Endgeräte und damit mit den geringsten Entwicklungs- und Pflegekosten erfüllt.

- **Eignung:** Da Kunden Bilder hochladen, die Gravurvorschau prüfen und anschließend bestellen, muss die Anwendung Daten serverseitig verarbeiten und speichern, was eine statische Website nicht leisten kann. Eine Mobile App oder Desktop-Anwendung könnte das ebenfalls, müsste aber vor der ersten Bestellung installiert werden.
- **Wartbarkeit:** Alle Rollen nutzen dieselbe Codebasis und werden über rollenbasierte Zugriffskontrolle (RBAC) getrennt. Updates und Pflege fallen dadurch nur einmal an, während Mobile App und Desktop-Anwendung je Plattform gepflegt werden müssen. Eine Erweiterung zur Progressive Web App ist später auf derselben Codebasis möglich.
- **Skalierbarkeit:** Mehr Kunden belasten nur den Server und nicht deren Geräte, sodass höhere Last zentral abgefangen wird. Für den Online-Betrieb genügt deshalb ein Deployment mit HTTPS statt einer Neuentwicklung.
- **Entwicklungsaufwand:** Es wird nur eine einzige Implementierung benötigt, die ohne Anpassung in allen aktuellen Browsern unter Windows, macOS, Linux, iOS und Android läuft, statt getrennter Versionen für iOS, Android und Desktop. Dabei fallen weder Store- noch Lizenzgebühren und im lokalen Betrieb auch keine Hostingkosten an.
- **Aktualität:** Responsive Design ist Standard der Webentwicklung und ersetzt eine eigene Mobilversion, wodurch deren Entwicklungskosten entfallen.

## 2.2 Programmiersprachen und Frameworks

| Option | Vorteile | Nachteile |
|---|---|---|
| **Python + FastAPI** | Schlank, Validierung und API-Doku integriert | Login und Admin-Oberfläche selbst umzusetzen |
| **Python + Django** | Admin, Login und ORM integriert, bewährt | Mehr Einarbeitung, für reine APIs zu umfangreich |
| **Node.js (Express, NestJS)** | Eine Sprache für Frontend und Backend | Weniger Bild- und KI-Bibliotheken als Python |
| **Java + Spring Boot** | Typsicher, performant, Standard in Großprojekten | Hoher Code- und Konfigurationsaufwand |
| **Frontend ohne Framework (HTML, CSS, JS)** | Kein Build-Prozess, keine Einarbeitung | Bei großem Umfang unübersichtlicher als React, Vue, Angular |

> **Behauptung:** Python mit FastAPI ist im Vergleich zu Django, Node.js und Spring Boot die beste Wahl, weil es Backend, Bildverarbeitung und KI-Anbindung mit dem geringsten Aufwand aus einer Technologie liefert. Das Frontend ohne React, Vue oder Angular spart zusätzlich Einarbeitung und Build-Prozess.

- **Eignung:** Weil jedes hochgeladene Foto automatisch für die Gravur aufbereitet wird, steht die Bildverarbeitung im Mittelpunkt, und dafür bietet Python ausgereiftere Bibliotheken als Node.js. FastAPI ist im Gegensatz zu Django gezielt für API-Backends ausgelegt und prüft und dokumentiert die Schnittstellen bereits ohne Erweiterungen.
- **Wartbarkeit:** FastAPI prüft alle Eingaben zentral an der Schnittstelle, sodass fehlerhafte Daten gar nicht erst in die Datenbank gelangen und spätere Korrekturen entfallen. Da Frontend und Backend nur über eine dokumentierte REST-Schnittstelle kommunizieren, lassen sich beide Teile unabhängig voneinander ändern und erweitern.
- **Skalierbarkeit:** Durch asynchrone Verarbeitung (ASGI) bedient FastAPI viele gleichzeitige Anfragen und lässt sich mit einer Client-Server-Datenbank und gemeinsamem Dateispeicher horizontal skalieren. Rechenintensive Bildverarbeitung kann bei Bedarf ausgelagert werden.
- **Entwicklungsaufwand:** Die Entwicklung baut auf bereits umgesetzten und getesteten Kernfunktionen auf, und die KI wird über vortrainierte Modelle ohne eigenes Training eingebunden. Gegenüber Spring Boot entsteht weniger Code- und Konfigurationsaufwand, und das Frontend ohne Framework spart im Vergleich zu React, Vue oder Angular Build-Prozess und Einarbeitung.
- **Aktualität:** Laut Stack Overflow Developer Survey 2025 zählt Python zu den meistgenutzten Programmiersprachen und FastAPI zu den deutlich wachsenden Web-Frameworks. Damit handelt es sich um aktuelle, aktiv weiterentwickelte Technologien, für die auch künftig Entwickler verfügbar sind.

## 2.3 Datenhaltung

| Option | Vorteile | Nachteile |
|---|---|---|
| **SQLite (eingebettet)** | Kein Server, Backup per Dateikopie | Ein Schreibzugriff gleichzeitig, nur ein Server |
| **PostgreSQL, MySQL** | Viele gleichzeitige Schreibzugriffe, mehrere Server möglich | Datenbankserver muss betrieben werden |
| **MongoDB (NoSQL)** | Flexibles Schema | Keine Fremdschlüssel, Integrität im Code |
| **Dateien (JSON, CSV)** | Keine Datenbank nötig, sehr einfach | Keine Integritätsprüfung, Konflikte bei gleichzeitigem Schreiben |
| **Cloud-Dienste (Supabase, Firebase)** | Schneller Start, Hosting inklusive | Nutzungsabhängige Kosten, Anbieterabhängigkeit, Daten bei Dritten |

> **Behauptung:** SQLite mit Dateisystem ist im Vergleich zu PostgreSQL/MySQL, MongoDB, JSON-Dateien und Cloud-Diensten die beste Wahl, weil sie relationale Datenintegrität ohne Datenbankserver und ohne laufende Kosten bietet und der Weg zu PostgreSQL offen bleibt.

- **Eignung:** Jede Bestellung gehört zu genau einem Kunden und verweist auf konkrete Produkte, die Daten sind also von Natur aus miteinander verknüpft. Eine relationale Datenbank bildet diese Beziehungen direkt ab und verhindert zum Beispiel doppelt registrierte E-Mail-Adressen, während dies bei MongoDB oder JSON-Dateien selbst im Code abgesichert werden müsste.
- **Wartbarkeit:** Das Datenmodell ist über SQLAlchemy zentral im Code definiert, sodass Änderungen an Tabellen nur an einer Stelle vorgenommen werden. Bilder und Produktionsdateien liegen in Standardformaten (PNG, JPEG, SVG, DXF) im Dateisystem, während die Datenbank nur den Dateipfad speichert und dadurch klein und leicht zu sichern bleibt.
- **Skalierbarkeit:** Laut SQLite-Dokumentation eignet sich SQLite für Websites unter 100.000 Aufrufen pro Tag (konservative Schätzung). Darüber erfolgt der Wechsel zu PostgreSQL oder MySQL über SQLAlchemy im Wesentlichen per Verbindungskonfiguration, sodass Investitionen erst anfallen, wenn das Wachstum es erfordert.
- **Entwicklungsaufwand:** Anders als bei PostgreSQL oder MySQL entfallen Installation, Betrieb und Hosting eines Datenbankservers, und im Gegensatz zu Supabase oder Firebase fallen keine nutzungsabhängigen Gebühren an.

- **Aktualität:** SQLite wird aktiv weiterentwickelt und zählt laut Stack Overflow Developer Survey 2025 zu den meistgenutzten Datenbanken. Zudem ist es in Android, iOS und den gängigen Browsern integriert, es handelt sich also um eine bewährte Technologie, die langfristig verfügbar bleibt.

## 2.4 Architektur

| Option | Vorteile | Nachteile |
|---|---|---|
| **Klassischer Monolith** | Schnell aufgesetzt, ein Deployment | Ohne Modulgrenzen riskante Änderungen |
| **Modularer Monolith** | Klare Module, geringer Betriebsaufwand | Skalierung nur als Ganzes |
| **Microservices** | Unabhängig entwickel- und skalierbar | Hoher Aufwand für Kommunikation und Betrieb |
| **Serverless (AWS Lambda)** | Keine Serververwaltung, nutzungsbasiert | Cloud-Abhängigkeit, Kaltstarts, lokal kaum testbar |

> **Behauptung:** Die Client-Server-Architektur als modularer Monolith ist im Vergleich zu klassischem Monolith, Microservices und Serverless die beste Wahl, weil sie strukturierter als der klassische Monolith, in Entwicklung und Betrieb günstiger als Microservices und anders als Serverless ohne Cloud-Abhängigkeit ist.

- **Eignung:** Kunden, Mitarbeiter und Admins greifen über den Browser auf denselben Datenbestand zu, deshalb übernimmt ein zentraler Server die Logik und prüft bei jeder Anfrage die Berechtigungen. Weil der Weg vom Upload bis zur fertigen Produktionsdatei ein zusammenhängender Prozess ist, würde eine Aufteilung in Microservices Entwicklungszeit kosten, ohne fachlichen Nutzen zu bringen.
- **Wartbarkeit:** Im Gegensatz zum klassischen Monolithen trennt die Schichtenarchitektur (Router, Services, Models) die Zuständigkeiten klar, wodurch Fehler schnell lokalisierbar sind und der Supportaufwand sinkt. Lose gekoppelte Module erlauben es, KI-Komponenten zu tauschen oder eine Mobile App zu ergänzen, ohne bestehende Module anzufassen.
- **Skalierbarkeit:** Mit PostgreSQL und gemeinsamem Dateispeicher lässt sich die Anwendung horizontal skalieren, und rechenintensive Module können ausgelagert werden, sodass Infrastrukturkosten erst bei Bedarf entstehen.
- **Entwicklungsaufwand:** Es gibt ein Projekt mit einem Deployment, und im lokalen Betrieb wird keine Cloud-Infrastruktur benötigt. Anders als bei Serverless ist die Anwendung vollständig lokal testbar, was Entwicklungszeit spart.
- **Aktualität:** Der modulare Monolith ist ein aktuell etablierter Architekturansatz, der klare Modulgrenzen mit dem geringen Betriebsaufwand einer einzelnen Anwendung verbindet. Einzelne Module lassen sich bei Bedarf später als eigene Dienste auslagern, der Weg zu Microservices bleibt also offen, ohne heute deren Kosten zu verursachen.